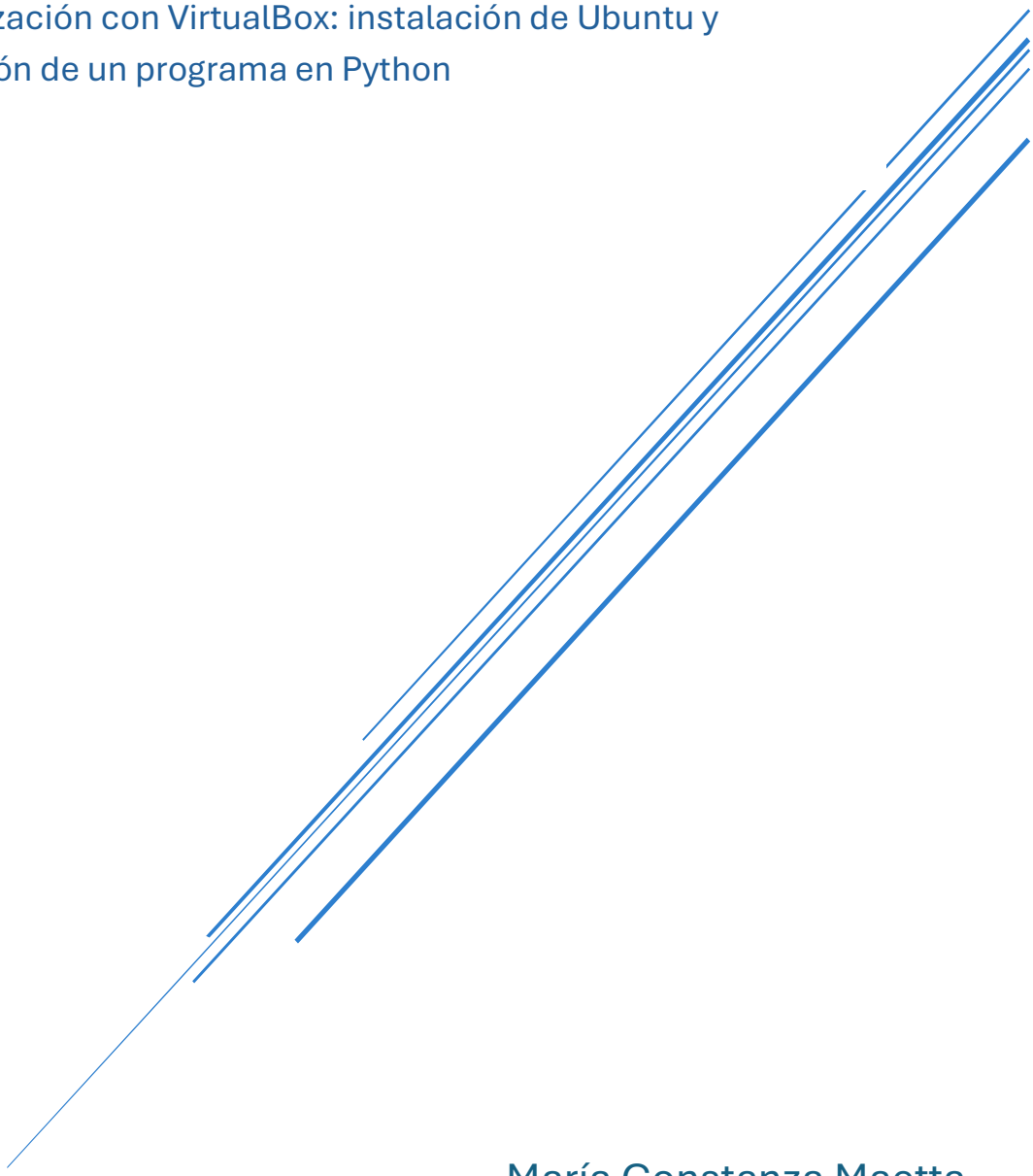
UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación
Arquitectura y Sistemas Operativos

Trabajo Práctico Integrador
Virtualización con VirtualBox: instalación de Ubuntu y ejecución de un programa en Python



María Constanza Maetta

8 de Noviembre de 2026

CONTENIDO

desarrollo del trabajo integrador de arquitectura y sistemas operativos 2
1. Introducción 2
2. Marco teórico 2
3. Caso práctico 3
4. Metodología utilizada 4
4.1. Verificación de la imagen ISO de Ubuntu 4
4.2. Creación y configuración de la máquina virtual 4
4.3. Instalación de Ubuntu 5
4.4. Ejecución del programa en Python 7
4.5. Ejecución del programa en un contenedor Docker 7
5. resultados obtenidos 9
6. conclusiones 9
7. bibliografía 10
8. anexos 10
Anexo A. Código fuente del programa 10
Anexo B. Dockerfile del programa. 10

DESARROLLO DEL TRABAJO INTEGRADOR DE ARQUITECTURA Y SISTEMAS OPERATIVOS

1. INTRODUCCIÓN

En este trabajo se utilizó Oracle VirtualBox para crear una máquina virtual con Ubuntu Desktop 24.04.5 LTS en una computadora con Windows. Se eligió la virtualización porque permite preparar un entorno de práctica con un sistema operativo distinto del anfitrión. En la formación en programación, esto facilita probar aplicaciones y familiarizarse con otras herramientas y sistemas operativos.

El objetivo fue configurar una máquina virtual, instalar Ubuntu y comprobar que un mismo programa en Python podía ejecutarse de dos formas: directamente en el sistema invitado y dentro de un contenedor Docker. Para comparar ambas ejecuciones, se utilizó un programa que calcula el promedio de tres tiempos de respuesta simulados.

2. MARCO TEÓRICO

La virtualización permite crear máquinas virtuales que utilizan los recursos de una computadora física. Cada máquina dispone de recursos virtuales, como memoria RAM, procesadores y almacenamiento, con los que puede ejecutar un sistema operativo y sus aplicaciones. El hipervisor administra el acceso de las máquinas virtuales a los recursos físicos disponibles.

Existen dos tipos principales de hipervisores. Los de tipo 1 se ejecutan directamente sobre el hardware, como VMware ESXi. Los de tipo 2 funcionan sobre un sistema operativo anfitrión, como Oracle VirtualBox. En esta práctica se utilizó VirtualBox sobre Windows para ejecutar Ubuntu como sistema operativo invitado.

El sistema anfitrión es aquel sobre el que se instala VirtualBox, mientras que el invitado funciona dentro de la máquina virtual. Esta organización permite utilizar distintos sistemas operativos en una misma computadora y preparar entornos de desarrollo y prueba. Los recursos asignados a las máquinas virtuales están limitados por la capacidad del equipo físico; por eso, al configurarlas, se debe considerar también lo que necesita el anfitrión para continuar funcionando.

Una máquina virtual puede utilizarse como entorno de pruebas aislado o sandbox: un espacio delimitado para experimentar con instalaciones y configuraciones. Sin embargo, el aislamiento no constituye una garantía absoluta de seguridad; el hipervisor también debe mantenerse protegido, ya que una vulnerabilidad en él puede afectar a las máquinas que administra.

Docker permite ejecutar aplicaciones en contenedores. Un contenedor es un proceso aislado con los archivos que necesita para funcionar. A diferencia de una máquina virtual,

que ejecuta un sistema operativo con su propio kernel, los contenedores comparten el kernel del sistema donde se ejecutan. El kernel es el núcleo del sistema operativo. Esta diferencia permite que los contenedores eviten la necesidad de iniciar un sistema operativo completo para cada aplicación.

En este trabajo se combinaron ambas tecnologías: VirtualBox ejecutó Ubuntu como invitado sobre Windows, y Docker funcionó dentro de Ubuntu. Por lo tanto, el contenedor compartió el kernel de Ubuntu y utilizó el intérprete de Python incluido en su imagen. La ejecución directa, en cambio, utilizó el intérprete instalado en Ubuntu.

La imagen Docker proporciona los archivos y la configuración utilizados para crear el contenedor. Para construirla se emplea un Dockerfile, un archivo de texto que describe las instrucciones de construcción. En esta práctica, FROM seleccionó la imagen base python:3.12-slim; WORKDIR estableció /app como directorio de trabajo; COPY incorporó tiempos_respuesta.py; y CMD definió el comando predeterminado para ejecutar el programa al iniciar el contenedor.

El uso del Dockerfile permite documentar la preparación del entorno de la aplicación y facilita repetir su construcción. La práctica comparó la ejecución directa del programa en Ubuntu con su ejecución en Docker mediante los mismos datos de entrada. Esta comparación permitió comprobar la coincidencia de los resultados para el caso ensayado; no constituyó una medición de rendimiento entre ambas tecnologías.

3. CASO PRÁCTICO

El caso práctico consistió en preparar un entorno Linux en una computadora con Windows para ejecutar un programa desde la terminal. Para ello, se creó en Oracle VirtualBox la máquina virtual Ubuntu-UTN-TPI, con 4 GB de memoria RAM, dos procesadores virtuales y un disco virtual de 40 GB. En ese disco se instaló Ubuntu Desktop 24.04.5 LTS.

Para comprobar el funcionamiento del entorno, se utilizó el programa tiempos_respuesta.py, que solicita tres tiempos de respuesta simulados en milisegundos, los guarda en una lista y muestra su promedio con dos decimales. Primero se ejecutó con el intérprete instalado en Ubuntu. Luego se empaquetó en una imagen Docker y se ejecutó en un contenedor dentro de la misma máquina virtual. En ambas pruebas se ingresaron los mismos valores y se compararon las salidas con un cálculo manual.

El repositorio del proyecto se encuentra disponible en:
https://github.com/cotimaetta/TPI-Virtualizacion-UTN

4. METODOLOGÍA UTILIZADA

4.1. VERIFICACIÓN DE LA IMAGEN ISO DE UBUNTU

Antes de crear la máquina virtual, se verificó la integridad de la imagen de instalación ubuntu-24.04.5.1-desktop-amd64.iso. Para ello, se calculó su huella SHA-256 en Windows PowerShell mediante el comando Get-FileHash y se comparó con el valor publicado por Ubuntu. Ambos valores coincidieron, lo que permitió comprobar que el archivo correspondía a la imagen oficial.

La huella obtenida se comparó con el valor publicado por Canonical (s. f.)

```
PS C:\Users\constanza.maetta\Downloads> Get-FileHash -Algorithm SHA256 ".\ubuntu-24.04.5.1-desktop-amd64.iso"

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          4DA4A0C9035DA8E68A59A838674F403F0A54472C78A83B4FB7F78D03588F85A7       C:\Users\constanza.maetta\Dow...
```

Figura 1. Cálculo de la huella SHA-256 de la imagen ISO de Ubuntu en Windows PowerShell.

4.2. CREACIÓN Y CONFIGURACIÓN DE LA MÁQUINA VIRTUAL

En Oracle VirtualBox se inició la creación de una máquina virtual denominada Ubuntu-UTN-TPI. Se seleccionó la imagen ISO de Ubuntu Desktop 24.04.5.1 LTS de 64 bits, verificada en el paso anterior. Se desactivó la instalación desatendida para configurar Ubuntu manualmente y registrar las decisiones tomadas durante el proceso.

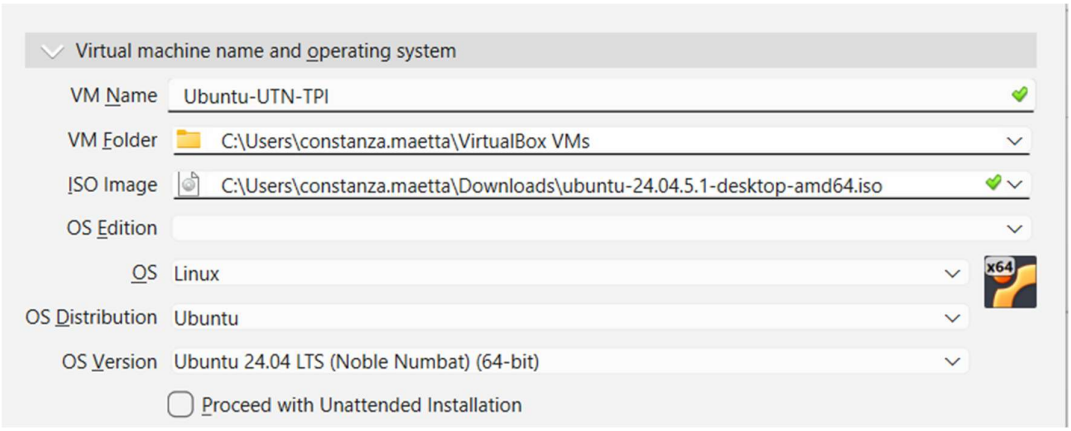


Figura 2. Selección de la imagen ISO y del modo de instalación en VirtualBox.

Se asignaron 4096 MB de memoria RAM y dos procesadores virtuales (vCPU) a la máquina virtual. Esta configuración se eligió para ejecutar el escritorio de Ubuntu y realizar las pruebas del programa en Python, dejando margen de recursos para Windows, el sistema anfitrión.

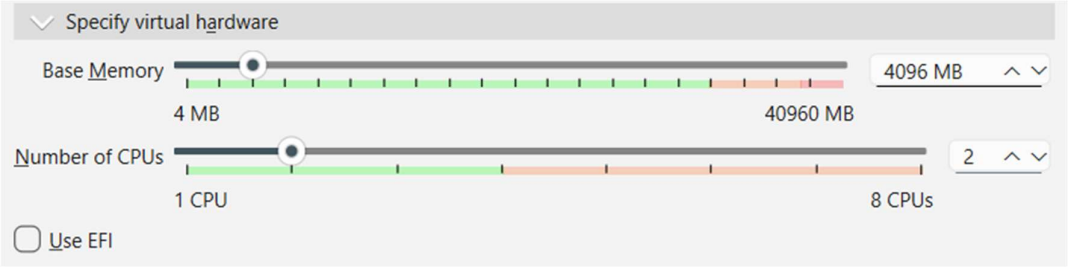


Figura 3. Asignación de memoria RAM y procesadores virtuales en VirtualBox.

Se creó un disco virtual de 40 GB en formato VDI para alojar Ubuntu, sus actualizaciones y los archivos de la práctica. Se utilizó la asignación dinámica para que el archivo del disco virtual aumentara de tamaño a medida que el sistema invitado escribiera datos.

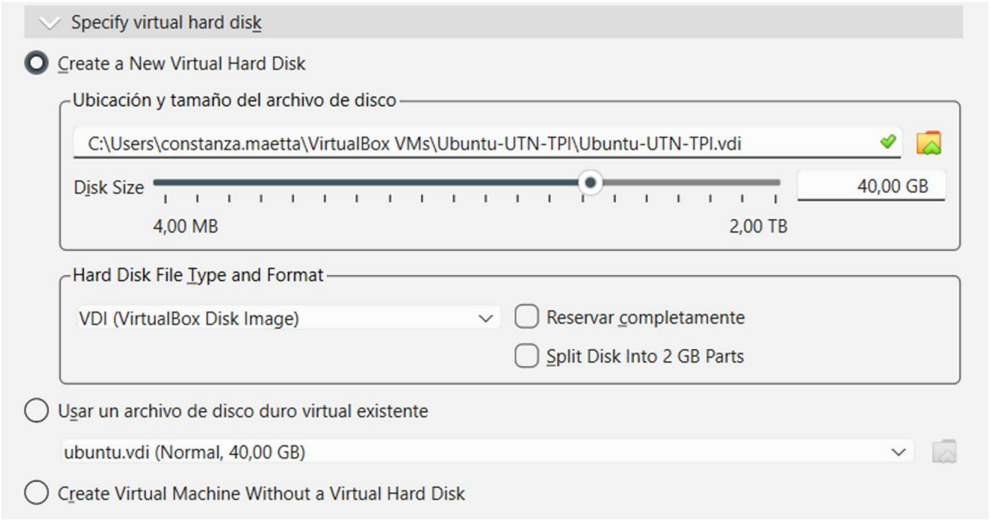


Figura 4. Configuración del disco virtual VDI de 40 GB con asignación dinámica.

4.3. INSTALACIÓN DE UBUNTU

Se inició la máquina virtual desde la imagen ISO verificada. El arranque desde este medio permitió acceder al asistente de instalación de Ubuntu Desktop.

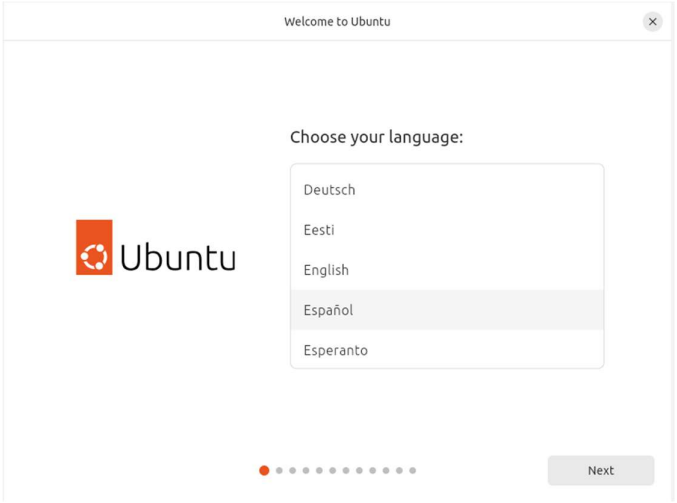


Figura 5. Pantalla inicial del asistente de instalación de Ubuntu Desktop en VirtualBox.

Se continuó la instalación sin conexión a Internet, utilizando la imagen ISO verificada previamente. En el asistente se seleccionó «Instalar Ubuntu» para continuar con la instalación en el disco virtual. Se eligió la selección predeterminada de aplicaciones porque la práctica se realiza desde la terminal y no requiere una suite de oficina. Se continuó sin instalar software privativo adicional. Se eligió el particionado automático del disco virtual creado para la máquina Ubuntu-UTN-TPI

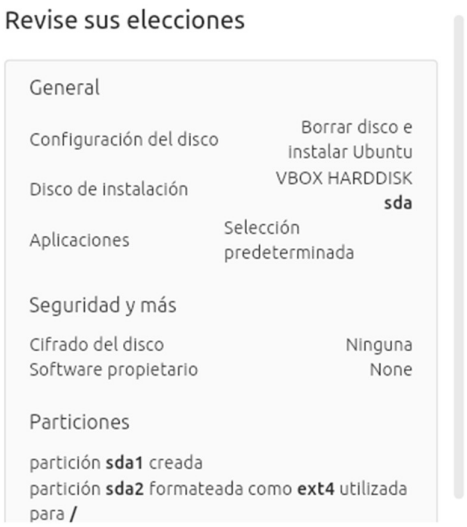


Figura 6. Resumen de instalación de Ubuntu en el disco virtual VBOX HARDDISK (sda).

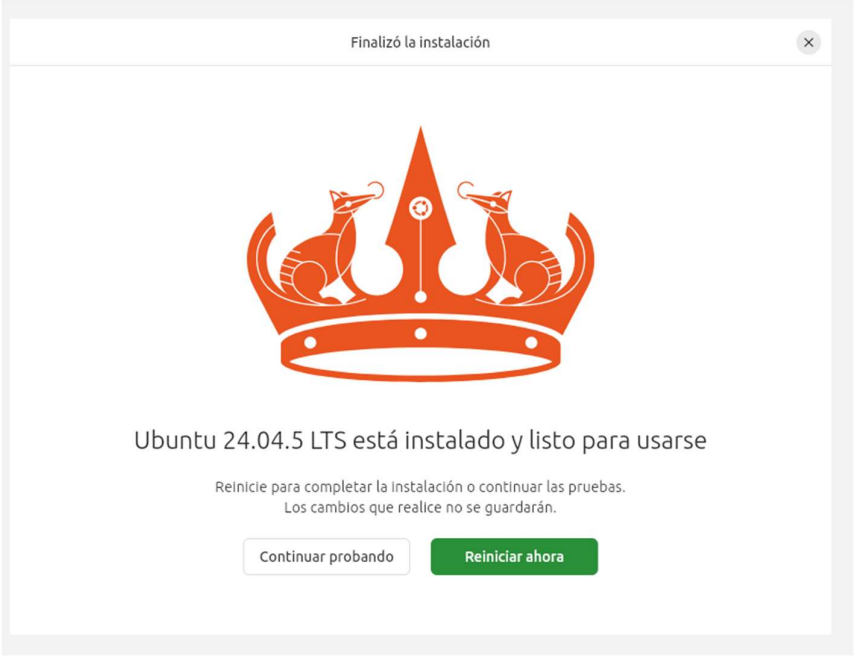


Figura 7. Confirmación de la instalación de Ubuntu 24.04.5 LTS en la máquina virtual.

Tras reiniciar la máquina virtual desde el disco virtual, se inició sesión en Ubuntu. Desde la terminal, se ejecutó lsb_release -ds para verificar que el sistema instalado era Ubuntu 24.04.5 LTS. Luego, con python3 --version, se comprobó que el intérprete Python 3.12.3 estaba disponible para la práctica.

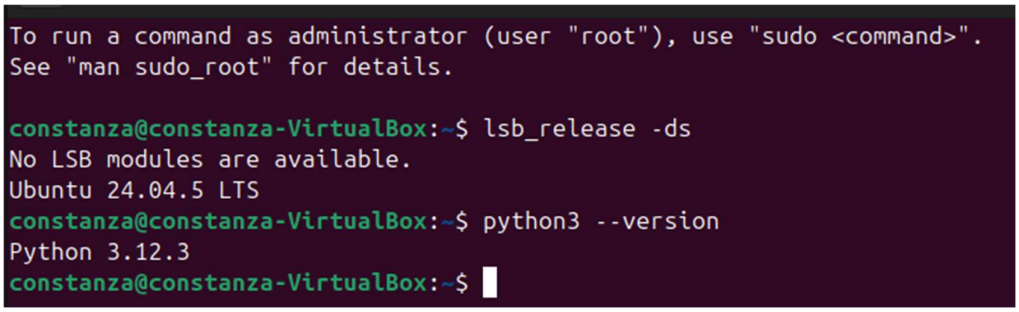


Figura 8. Verificación de las versiones de Ubuntu y Python en la máquina virtual.

4.4. EJECUCIÓN DEL PROGRAMA EN PYTHON.

Se creó el archivo tiempos_respuesta.py con el editor Nano dentro de la máquina virtual. El código fuente completo se incluye en el Anexo A. El programa solicita tres tiempos de respuesta simulados en milisegundos, los almacena en una lista y calcula su promedio. Se ejecutó desde la terminal con python3 tiempos_respuesta.py. Para verificar el cálculo, se ingresaron los valores 120, 150 y 180 ms. El programa mostró un promedio de 150.00 ms, que coincidió con el resultado esperado.

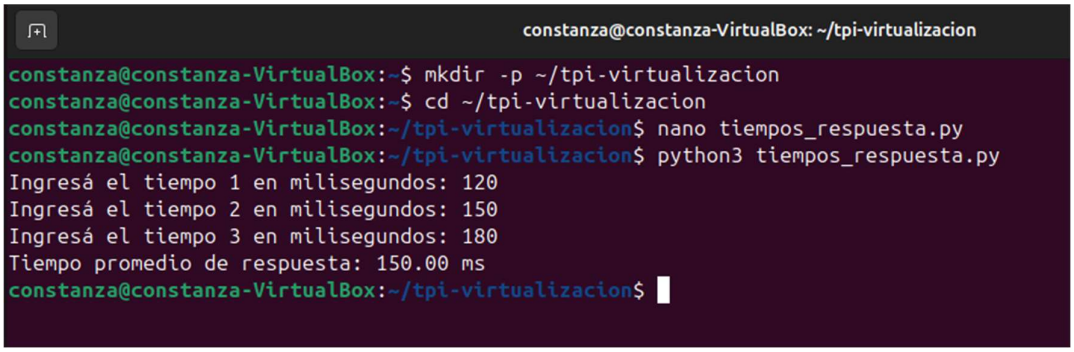


Figura 9. Ejecución del programa en Python y comprobación del promedio con datos simulados.

4.5. EJECUCIÓN DEL PROGRAMA EN UN CONTENEDOR DOCKER

Se instaló Docker en Ubuntu mediante el paquete docker.io. Se verificó la versión instalada con docker --version y se ejecutó sudo docker run --rm hello-world. Docker descargó la imagen de prueba y ejecutó un contenedor que mostró el mensaje «Hello from Docker!».

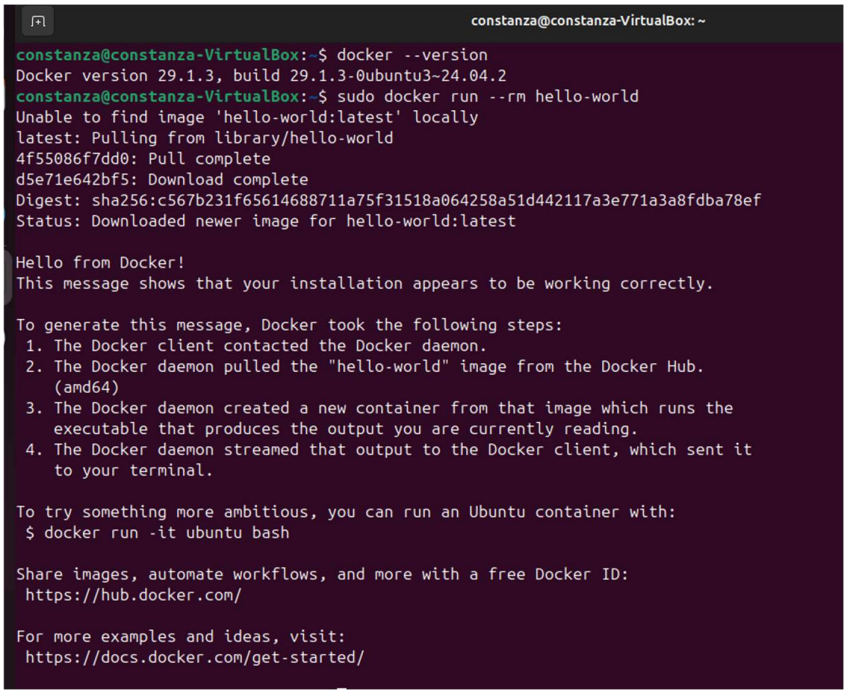


Figura 10. Verificación de Docker 29.1.3 mediante la ejecución del contenedor de prueba hello-world.

Se creó un Dockerfile que utiliza la imagen base python:3.12-slim, copia el archivo tiempos_respuesta.py y define cómo ejecutarlo al iniciar el contenedor. El contenido completo del archivo se incluye en el Anexo B. Se construyó la imagen tiempos-respuesta:1.0 con sudo docker build -t tiempos-respuesta:1.0 . y se verificó su disponibilidad con sudo docker image ls tiempos-respuesta.

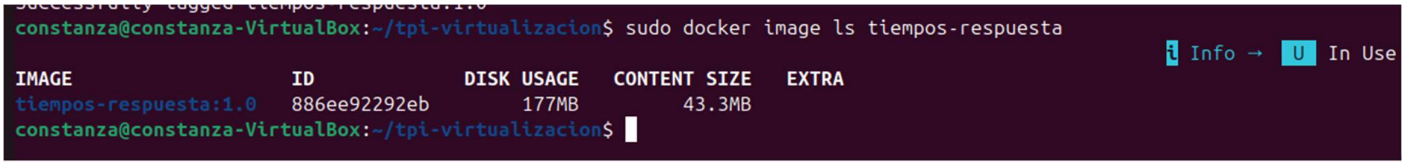


Figura 11. Verificación de la imagen tiempos-respuesta:1.0 tras su construcción.

Se ejecutó el programa en un contenedor creado a partir de la imagen tiempos-respuesta:1.0 mediante el comando sudo docker run --rm -it tiempos-respuesta:1.0. Se ingresaron los mismos tres valores simulados que en la prueba directa en Ubuntu: 120, 150 y 180 ms. El programa mostró un promedio de 150.00 ms, que coincidió con el resultado de la ejecución anterior.

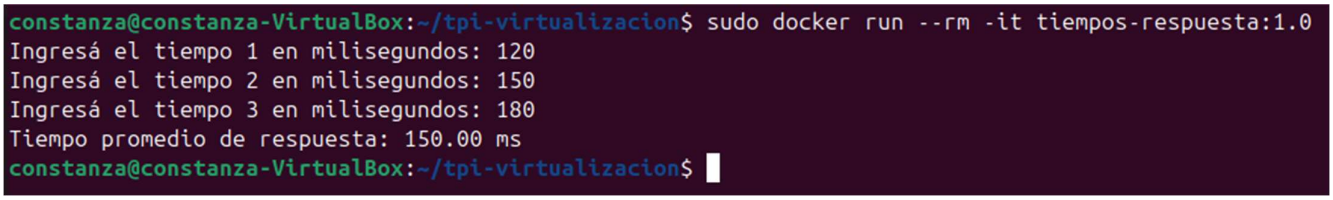


Figura 12. Ejecución interactiva de tiempos_respuesta.py en un contenedor Docker.

5. RESULTADOS OBTENIDOS

Se instaló Ubuntu 24.04.5 LTS en VirtualBox y se comprobó que la máquina virtual podía iniciar desde su propio disco virtual. Una vez iniciada la sesión, se verificó desde la terminal que Python 3.12.3 estaba disponible (Figura 8).

Se ejecutó tiempos_respuesta.py en dos entornos: directamente en Ubuntu con Python 3.12.3 (Figura 9) y en un contenedor creado a partir de la imagen tiempos-respuesta:1.0 (Figura 12). En ambas ejecuciones se ingresaron los mismos valores simulados, 120, 150 y 180 ms. El programa mostró un promedio de 150.00 ms en los dos casos, que coincidió con el cálculo manual. La comparación verificó que el programa produjo la misma salida en ambos entornos para los datos ensayados.

6. CONCLUSIONES

La práctica permitió comprender cómo VirtualBox asigna memoria, procesadores y almacenamiento a un sistema invitado. Al instalar Ubuntu en un disco virtual y ejecutar un programa en Python desde su terminal, se comprobó que la máquina virtual podía utilizarse como entorno de práctica independiente de Windows.

La práctica también permitió distinguir dos formas de preparar un entorno de ejecución. VirtualBox proporcionó una máquina virtual con Ubuntu como sistema operativo invitado. Dentro de ese Ubuntu, Docker ejecutó el programa en un contenedor que comparte el kernel del invitado y utiliza el intérprete de Python incluido en la imagen. El uso de un Dockerfile permitió dejar por escrito cómo construir la imagen y reproducir la ejecución del programa.

Durante los primeros intentos de arranque, la pantalla quedó en negro y apareció un mensaje del controlador gráfico vmwgfx. Luego se logró acceder al asistente y completar la instalación. La experiencia mostró la importancia de registrar los mensajes de error y comprobar el funcionamiento de cada etapa antes de continuar. Como mejora, se podría ampliar el programa para validar las entradas y probar el cálculo con distintos conjuntos de datos.

7. BIBLIOGRAFÍA

Canonical. (s. f.). *Sumas de comprobación SHA-256 de Ubuntu 24.04 LTS*. https://releases.ubuntu.com/24.04/SHA256SUMS

Microsoft. (s. f.). *Get-FileHash*. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash

Docker. (s. f.-a). Dockerfile overview. Docker Docs. https://docs.docker.com/build/concepts/dockerfile/

Docker. (s. f.-b). What is a container? Docker Docs. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Oracle. (s. f.). About Oracle VirtualBox. Oracle VirtualBox User Guide. https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/Introduction.html

Red Hat. (2023, 3 de enero). What is a hypervisor? https://www.redhat.com/en/topics/virtualization/what-is-a-hypervisor

8. ANEXOS

ANEXO A. CÓDIGO FUENTE DEL PROGRAMA

A continuación se presenta el contenido del archivo tiempos_respuesta.py, ejecutado con Python 3.12.3 dentro de la máquina virtual. El programa recibe tres tiempos de respuesta simulados en milisegundos y muestra su promedio con dos decimales.

```
tiempos = []
for numero in range(1, 4):
    tiempo = float(input(f"Ingresá el tiempo {numero} en milisegundos: "))
    tiempos.append(tiempo)
promedio = sum(tiempos) / len(tiempos)
print(f"Tiempo promedio de respuesta: {promedio:.2f} ms")
```

ANEXO B. DOCKERFILE DEL PROGRAMA.

El siguiente Dockerfile se utilizó para construir la imagen tiempos-respuesta:1.0 a partir de python:3.12-slim.

```
FROM python:3.12-slim
WORKDIR /app
COPY tiempos_respuesta.py .
CMD ["python", "tiempos_respuesta.py"]
```